I.Introduction

1.1 Purpose of the System:

The Furzefield Leisure Center (FCL) this requires a highly reliable and user-friendly software solution to manage group exercise booking. The organization has outlined the need for an automated reservation platform to efficiently handle class capacities, member schedules, reviews, and detailed financial report. The project delivers a robust java-based solution that successfully digitizes the administrative responsibilities of FLC.

1.2 Scope:

This report documents the design, development, and testing phases of the Java application handling the group exercise appointment. The application simulates the timetable and booking ecosystem across an 8-weekend period. It handles fundamental capabilities including, but not limited to, maintaining course capacities(minimum and maximum attendees), enforcing temporal collision constraints (avoiding duplication member booking for the same time slot), capturing user feedback/rating, and generating data-driven analytical reports.

1.3 Technologies and Environment:

Core Language: Java (Standard Edition, object-oriented)
Testing: JUnit for unit testing
Version Control: Git & GitHub
Design: UMLet for class diagrams

2. System Requirements Specifications

The system works based on a set of business rules defined by Furzefield Leisure Centre, and these rules must be followed for it to function correctly.

2.1 Functional Requirements

The system provides the following main features:

- Timetable nitialization: The system creates a schedule for 8 weekends (Saturday and Sunday). Each day has three sessions: Morning, Afternoon, and Evening. In total, this gives 48 lessons across the 8 weeks.
- Exercise Types: The system includes different types of classes such as Yoga, Zumba, Aquacise, and Box Fit.
- Capacity Limits: Each lesson can have a maximum of 4 members to keep the class size small and manageable.
- Booking Conflict Check: A member cannot book two lessons that happen at the same time. The system checks this before confirming a booking.
- Cancellation and Changes: Members can cancel their bookings, and lessons can also be changed if needed.
- Rating System: After attending a lesson, members can give a rating from 1 (Very dissatisfied) to 5 (Very satisfied), and they can also leave a comment if they want.
- Reports and Analysis: The system can generate monthly reports (every 4 weeks), including:
  - How many people attended each lesson
  - The average rating for each type of class
  - Which lesson generated the most revenue

2.2 Non-Functional Requirements

- Extensibility: The system is built using Object-Oriented Programming principles (like encapsulation and inheritance), which makes it easier to add new features later, such as payment options or a graphical user interface (GUI).
- Testability: The system should be easy to test using automated tools to make sure everything works correctly.

3. System Design and Architecture

The system is designed to keep things simple and well-organised by reducing dependencies between components (low coupling) and making sure each part has a clear responsibility (high cohesion). The structure reflects real-world interactions between members, lessons, and bookings.

3.1 Architectural Structure

The system follows a modular structure where different parts handle specific responsibilities:

- Main Controller (BookingSystem.java):This is the main class that controls how the system works. It handles user input from the console, manages the timetable, processes bookings, and generates reports.
- Model Classes (com.flc.model): This package contains the main entities of the system:
  - Lesson
  - Member
  - Booking
  - Review

  Each class is responsible for its own data and behaviour, following encapsulation principles.
- Timetable (com.flc.timetable.Timetable): This class is responsible for creating and managing the lesson schedule. It organises lessons across different days, times, and weeks.

3.2 Coupling and Cohesion

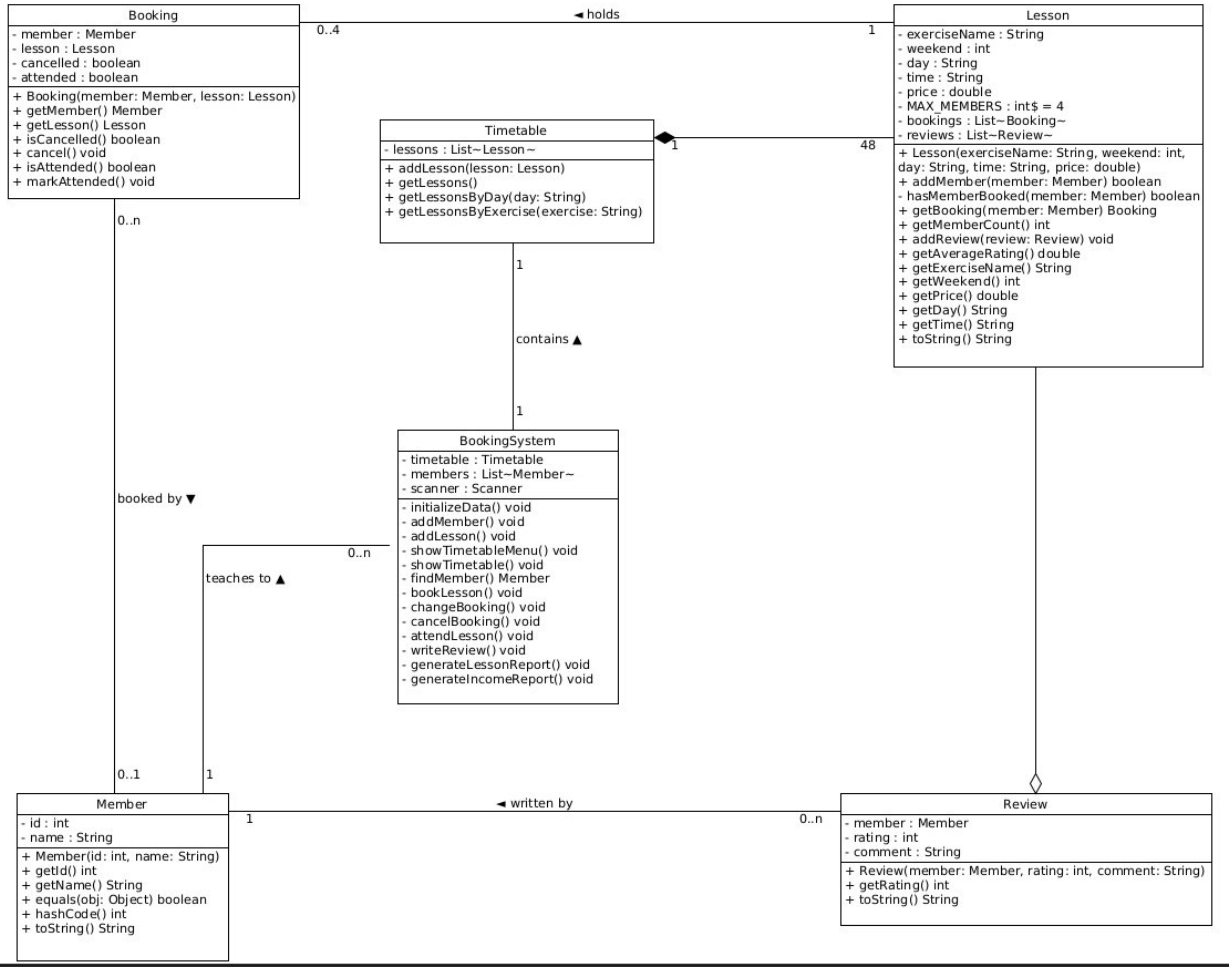
The system is designed so that each class focuses on one responsibility, while keeping dependencies between classes minimal.

For example, instead of storing duplicate information, the Booking class holds references to both a Member and a Lesson. This means:

- Data is not repeated unnecessarily
- Relationships between objects are clear
- It becomes easier to generate reports by navigating these object links

This approach improves the overall structure of the system and makes it easier to maintain and extend in the future.

4. UML Class Diagram:



5. Core Implementation Details

The implementation follows the system design and translates it into working Java code. Each part of the system is implemented as a class, and together they handle bookings, members, lessons, and reporting.

5.1 The Lesson Class

The Lesson class is located in the com.flc.model package and represents a single lesson in the system.

- Attributes:
  It includes properties such as lessonId, exerciseName (e.g., "Yoga", "Box Fit"), price, and time-related details (week, day, timeSlot). It also stores lists of Booking and Review objects.
- Core Functionality:
  The class enforces a maximum capacity of 4 members. When a new booking is added, the system checks the number of existing bookings. If the lesson is already full, the booking is rejected and the method returns false.

5.2 The Member Class

The Member class is a simple data class used to store information about users.

- It includes memberId, firstName.
- Each member is uniquely identified and used when creating bookings and tracking activity in the system.

5.3 The Booking and Review Classes

- Booking.java: This class links a Member to a Lesson. It also tracks the status of the booking using values such as BOOKED, ATTENDED, CHANGED, and CANCELLED.
- Review.java: This class stores feedback for lessons. It includes a rating (1 to 5) and is linked to a specific lesson. These ratings are later used to calculate average scores for reports.

5.4 The BookingSystem Controller

The BookingSystem.java class acts as the main controller of the system.

- It initializes the system with sample data (for example, creating 48 lessons).
- It provides a command-line interface (CLI) for user interaction. Input errors are handled using Scanner and exception handling to prevent crashes.
- It checks for booking conflicts by making sure a member cannot book two lessons at the same time.
- It generates monthly reports by processing the 8-week schedule in two 4-week sections. These reports include information such as total revenue and lesson attendance.

6. Quality Assurance & Testing

The system was tested to make sure it works correctly and meets all the required rules. Since the application relies on strict logic (like booking limits and time conflicts), unit testing was done using the JUnit 5 framework.

6.1 Objective

The main goal of testing is to make sure the system behaves correctly and to avoid breaking existing features when changes are made to the code.

6.2 The LessonTest Suite

The LessonTest class focuses on testing the behaviour of the Lesson class, especially around capacity and ratings.

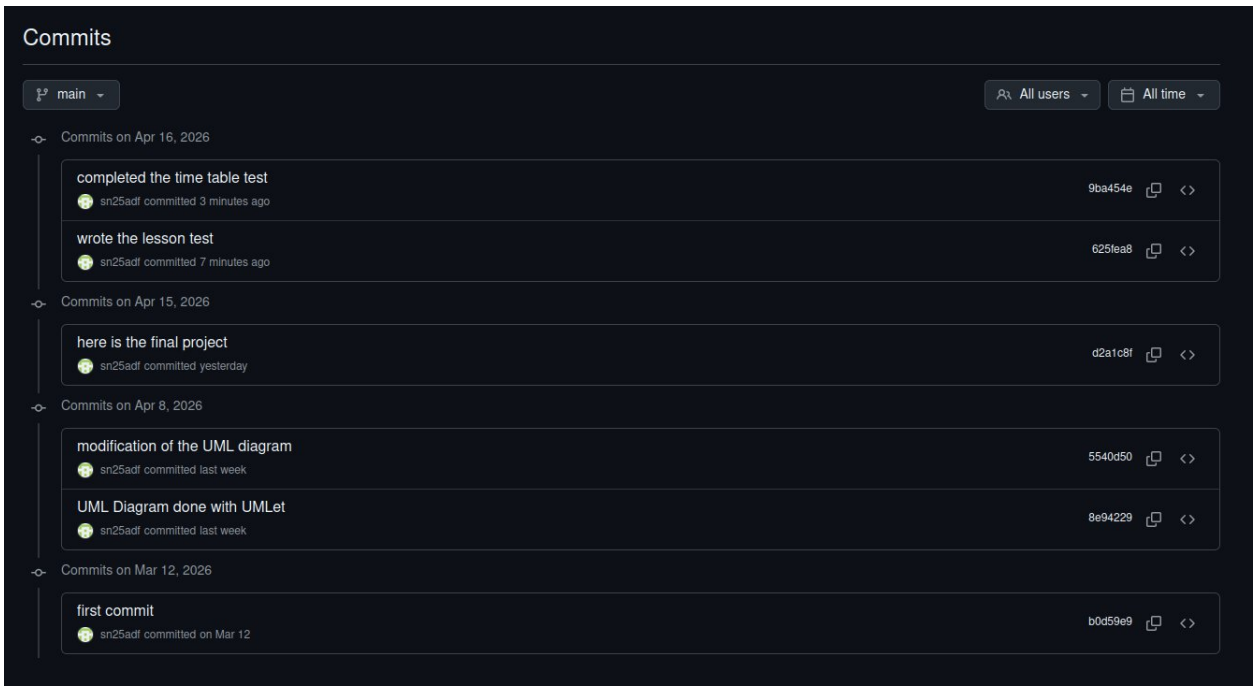
- testAddBooking(): This test checks that bookings can be added successfully up to the maximum limit of 4 members.
- testCapacityExceeded():
  This test tries to add a 5th booking to a lesson. The system correctly rejects it, confirming that the capacity limit is enforced.
- testReviewAveraging():
  This test checks that the average rating is calculated correctly. It also ensures the system does not crash (e.g., division by zero) when there are no reviews.

6.3 The TimetableTest Suite

The TimetableTest class focuses on verifying that the timetable is generated correctly.

- testGenerationValidity():
  This test checks that the timetable contains exactly 48 lessons
- It also verifies that all exercise types (Yoga, Zumba, Aquacise, and Box Fit) are properly distributed across the timetable.

7. Version Control Log



8. Summary

The Furzefield Leisure Centre Group Exercise Booking System is a complete and well-structured Object-Oriented solution that meets the initial requirements. It handles important constraints such as booking limits and time conflicts effectively, and provides useful reports based on system data.

The system is also designed to be extendable, making it easier to add new features in the future. In addition, the use of JUnit testing helps ensure that the system works correctly and reduces the risk of errors when changes are made.